Resumen Maestro: Lectura 5 — Eliminación en BST y Preservación de Invariantes

> **Curso:** CC232 - Algoritmos y Estructuras de Datos (UNI)
> **Cátedra:** Prof. César Lara Avila
> **Semana:** 05 — Eliminación en Árboles Binarios de Búsqueda
> **Archivo Original:** Lectura5.md

1. La Cadena Retrospectiva y Motivación de Ingeniería

```
problema -> operación dominante -> ADT -> representación -> invariante -> splice -> complejidad O(h)
```

En la Semana 4 construimos el BST para acelerar la búsqueda y la inserción basándonos en el orden de las claves. Cuando insertamos, el proceso es trivial a nivel estructural: una búsqueda fallida termina en una referencia `null` y conectamos allí una nueva hoja.

En la Semana 5 aparece el verdadero desafío: **¿cómo retirar una clave sin corromper la jerarquía?** Eliminar es sustancialmente más delicado que insertar. El nodo que deseamos retirar puede estar actuando como puente de uno o dos subárboles completos. Si simplemente desconectamos el enlace del padre (`p.left = null`), provocamos una **pérdida de alcanzabilidad (dejar nodos huérfanos en memoria que debían permanecer en el árbol)**. La eliminación exige localizar, comprobar existencia real, clasificar por grado y reconstruir los enlaces preservando simultáneamente 5 invariantes formales.

2. La Quíntupla Estructural Canónica

1. **ADT:** Conjunto dinámico ordenado con soporte de eliminación: `boolean remove(int x)`.
2. **Representación:** Árbol binario enlazado explícito. Cada nodo almacena `int x`, `Node left`, `Node right` y `Node parent`. El árbol mantiene `private Node root` y `private int n`.
3. **Invariantes Simultáneos Obligatorios:**
   - *Invariante Global de Orden BST:* Para cada nodo u, **todas** las claves del subárbol izquierdo son menores que $u.x$, y **todas** las del subárbol derecho son mayores ($\text{izq} < u.x < \text{der}$ sobre subárboles completos, no solo hijos inmediatos).
   - *Integridad Bidireccional de Punteros:* Para todo hijo existente: `u.left.parent == u` y `u.right.parent == u`.
   - *Contrato de Raíz:* Si el árbol no está vacío ($n > 0$), `root != null` y `root.parent == null`. Si $n == 0$, `root == null`.
   - *Invariante de Conteo:* `n` debe coincidir exactamente con el número de nodos alcanzables navegando desde `root`.
   - *Unicidad:* No se admiten claves duplicadas.
4. **Mecanismos de Reparación Estructural:**
   - `splice(Node u)` para nodos con a lo más un hijo (≤ 1 hijo).

- Sustitución por **sucesor inorder (el nodo con la menor clave dentro del subárbol derecho)** para nodos con dos hijos.

5. **Complejidad Asintótica:** $O(h)$, dominada estrictamente por el costo de localización, donde h es la altura del árbol ($O(\log n)$ en árboles balanceados, pero potencialmente $O(n)$ en árboles degenerados).

---

3. La Operación Atómica: `splice(Node u)`

`splice(Node u)` es la herramienta de bajo nivel que reconecta la estructura retirando físicamente un nodo.

Precondición Innegociable de `splice`:

> **IMPORTANTE**
>
> El nodo u debe tener **a lo más un hijo**: `u.left == null || u.right == null`. **Está terminantemente prohibido llamar a `splice` sobre un nodo con 2 hijos.**

```
Caso Un Hijo: Eliminar u=50 (con hijo superviviente s=55 y padre p=60)

        60 (p)                          60 (p)
       /                               /
     50 (u)          ======>         55 (s)
       \
        55 (s)
```

Mecánica Interna de `splice`:

1. **Determinar el hijo superviviente (s):**

   ```
   Node s = (u.left != null) ? u.left : u.right;
   ```

   (Si u es hoja, $s = \text{null}$; si tiene un hijo, s es esa referencia).

2. **Reconectar con el padre ($p = u.parent$):**
   - Si u es la raíz (`u == root`): la nueva raíz pasa a ser s (`root = s`). Si $s \neq \text{null}$, se anula su padre (`s.parent = null`).
   - Si u no es raíz: se determina qué enlace de p apuntaba a u (`if (p.left == u) p.left = s; else p.right = s;`).

3. **Restaurar enlace hacia arriba:**
   - Si $s \neq \text{null}$, se actualiza `s.parent = p`.

Código Canónico de `splice(Node u)`:

```
private void splice(Node u) {
    Node s = (u.left != null) ? u.left : u.right;
    Node p = u.parent;

    if (u == root) {
        root = s;
        if (s != null) {
            s.parent = null;
        }
    } else {
        if (p.left == u) {
            p.left = s;
        } else {
            p.right = s;
        }

        if (s != null) {
            s.parent = p;
        }
    }
}
```

¿Por qué `splice` no modifica `n`?

Separación de responsabilidades: `splice` solo reconecta punteros en tiempo constante $O(1)$. En el caso de dos hijos, `splice` se reutiliza sobre un nodo auxiliar sin que signifique una eliminación lógica independiente. Por lo tanto, **`n--` se ejecuta únicamente dentro de `remove(x)`**.

---

4. El Caso Crítico: Eliminación con Dos Hijos

Cuando u tiene dos hijos (`u.left != null && u.right != null`), no existe un único hijo superviviente s. Si conectáramos la izquierda o la derecha al padre, el otro subárbol quedaría huérfano.

```
              50 (u)  <-- Nodo a eliminar (tiene 2 hijos)
             /      \
           30        80
                    /  \
                  60    90
                 /
               55 (w: Sucesor inorder = mínimo subárbol derecho)
                 \
                  57 (hijo derecho de w)
```

La Estrategia en Dos Tiempos:

1. **Localizar el sucesor inorder (w):** El sucesor inorder es el nodo con la clave inmediatamente superior a $u.x$. Como u tiene subárbol derecho, su sucesor es el **mínimo del subárbol derecho**:

```
Node w = u.right;
while (w.left != null) {
    w = w.left;
}
```

2. **Propiedad Geométrica Fundamental:** Al terminar el bucle, sabemos con certeza absoluta que `w.left == null`. Si tuviera hijo izquierdo, ese hijo sería menor que w, contradiciendo que w sea el mínimo. Por tanto, w tiene a lo más un hijo (su hijo derecho, si existe) y **cumple la precondición de** `splice`.
3. **Eliminación Lógica frente a Física:**
   - *Eliminación Lógica:* Copiamos el contenido: `u.x = w.x;`. El nodo físico u no cambia de dirección en memoria ni altera sus punteros `left`, `right` ni `parent`.
   - *Eliminación Física:* Retiramos el nodo duplicado residual llamando a `splice(w)`. Si w tenía un hijo derecho (como el 57 en el diagrama), `splice` lo engancha correctamente como hijo de su abuelo (el 60).
4. **Finalización:** `n--` y retornar `true`.

5. Algoritmo Maestro: `remove(int x)`

```
public boolean remove(int x) {
    // 1. LOCALIZACIÓN
    Node u = findLast(x);

    // 2. COMPROBACIÓN DE EXISTENCIA
    if (u == null || u.x != x) {
        return false; // Árbol vacío o clave ausente: no mutar nada
    }

    // 3. CLASIFICACIÓN Y MODIFICACIÓN
    if (u.left == null || u.right == null) {
        // Casos: Hoja (0 hijos) o Un Solo Hijo
        splice(u);
    } else {
        // Caso: Dos Hijos
        Node w = u.right;
        while (w.left != null) {
            w = w.left;
        }
        u.x = w.x;  // Eliminación lógica (copia de valor)
        splice(w);  // Eliminación física del sucesor
    }

    // 4. ACTUALIZACIÓN DEL ADT
    n--;
    return true;
}
```

6. Contrastes Conceptuales Clave

A. Localización frente a Modificación

- **Localizar ($O(h)$):** Descender por el árbol con `findLast` o buscar el sucesor siguiendo enlaces `left`. Es la operación que consume tiempo y depende de la forma del árbol.
- **Modificar ($O(1)$):** Asignar 3 o 4 referencias en `splice` o copiar un entero (`u.x = w.x`). Las modificaciones en memoria son instantáneas una vez conocidos los punteros.

B. Eliminación Lógica frente a Eliminación Física

- *Lógica:* La clave x desaparece del conjunto cuando se sobrescribe `u.x = w.x`.
- *Física:* El bloque de memoria del nodo redundante se desacopla del grafo mediante `splice(w)` para que el recolector de basura (*Garbage Collector*) de la JVM lo libere.

C. Correctitud Estructural frente a Eficiencia de la Forma

Un BST puede ser 100% correcto (cumplir orden, padres válidos, tamaño coherente e inorder ordenado) y, sin embargo, ser desastroso en rendimiento si su altura degenera en una cadena lineal ($h = n \to O(n)$). La eliminación en BST simple preserva la correctitud, pero no controla la altura.

---

7. Trampas Fijas de Examen (Catálogo de Errores UNI)

1. **Confiar en que `findLast(x) != null` implica que la clave existe:** Si buscas 52 en un árbol donde el último nodo es 55, `findLast(52)` retorna el nodo 55. Si no validas `u.x == x`, eliminarás un elemento inocente.
2. **Decrementar `n` dos veces en el caso de dos hijos:** Pensar que porque intervienen dos nodos (u y w), se resta dos veces. Solo se eliminó un elemento del conjunto lógico.
3. **Creer que el sucesor siempre es una hoja:** El sucesor inorder nunca tiene hijo izquierdo, pero **sí puede tener hijo derecho**. `splice` debe reconectar ese hijo derecho con el abuelo.
4. **Olvidar actualizar `s.parent`:** Asignar `p.left = s` pero olvidar `s.parent = p`. El árbol se verá perfecto si se recorre de arriba hacia abajo, pero se corromperá ante cualquier algoritmo que ascienda por los padres.
5. **Creer que el recorrido `inorder` valida la estructura al 100%:** Un árbol con punteros `parent` corruptos o con `n` desfasado puede seguir imprimiendo una secuencia ordenada en `inorder`.

---

8. Tabla Resumen de Complejidad Asintótica

Operación	Mejor Caso	Caso Promedio	Peor Caso (Árbol Degenerado)	Justificación Física / Algorítmica
`splice(u)`	$O(1)$	$O(1)$	$O(1)$	Reasigna un número constante de referencias en memoria.
`findLast(x)`	$O(1)$	$O(\log n)$	$O(h) = O(n)$	Recorre un único camino descendente acotado por la altura h.
Buscar Sucesor	$O(1)$	$O(\log n)$	$O(h) = O(n)$	Desciende por la rama izquierda del subárbol derecho ($< h$).
`remove(x)`	$O(1)$	$O(\log n)$	$O(h) = O(n)$	Dominado enteramente por la localización descendente.

9. Comprueba tu Comprensión (Verificación y Solucionario de Trazado)

1. **Estado:** Tienes un árbol con raíz 40, hijo derecho 60, hijo izquierdo de sesenta 50, e hijo derecho de cincuenta 55 (`40 -> right: 60 -> left: 50 -> right: 55`). Si ejecutas `remove(50)`, ¿quién es u, quién es s, quién es p, y qué punteros exactos cambian?

- **Identificación:**
  - u es el nodo a retirar: `u = 50`.
  - s es el hijo superviviente: `s = 55` (único hijo, pues `50.left == null` y `50.right != null`).
  - p es el padre estructural: `p = 60` (`u.parent = 60`).
- **Punteros que cambian en memoria:**
  - Como u es hijo izquierdo de p (`p.left == u`): `p.left = s` (es decir, `60.left = 55`).
  - Como $s \neq \text{null}$: `s.parent = p` (es decir, `55.parent = 60`).
  - **Aclaración clave sobre `u.parent` y `u.right`:** En Java (y en la implementación canónica de Lara), `splice` **NO necesita limpiar** `u.parent = null` **ni** `u.right = null`. El recolector de basura (*Garbage Collector*) de la JVM detecta que u ya no es alcanzable desde la raíz (`root`) y libera su memoria automáticamente. Solo mutan `p.left` y `s.parent`. Luego, en `remove` se ejecuta `n--`.

2. **Invariante:** Si en el caso de dos hijos, en lugar de copiar la clave `u.x = w.x`, intentaras mover físicamente el nodo w a la posición de u, ¿cuántos punteros tendrías que reconectar y qué riesgo de punteros nulos aparece?

- **Verificación y Análisis:**
  - **Estimación correcta:** Se tendrían que reasignar entre 6 y 8 punteros (`w.parent`, `w.left`, `w.right`, el puntero del padre de u hacia w, los punteros inversos de los dos hijos de u hacia w, y desconectar a w de su padre original reconectando su posible hijo derecho).
  - **Riesgo crítico (Trampa de examen):** Si el sucesor w es el hijo derecho directo de u (`u.right == w`), al hacer ingenuamente `w.right = u.right` ¡creas una referencia circular sobre sí mismo (`w.right = w`) que genera un bucle infinito en cualquier recorrido! Además, si u era raíz, intentar leer `u.parent.left` arroja `NullPointerException`.
  - **Conclusión de diseño:** Copiar la clave `u.x = w.x` y aplicar `splice(w)` reduce 8 mutaciones peligrosas a una copia de entero y un `splice` limpio de 2 punteros.

3. **Frontera:** Si el árbol tiene un único nodo (`n = 1`) y llamas a `remove(root.x)` , ¿qué ramas de `splice` se ejecutan y cómo quedan `root` y `n` ?

- **Verificación:**
  - **Exacto:** En `splice(root)` : como es hoja, $s = \text{null}$ y $p = \text{null}$.
  - Entra a la rama `if (u == root)` : se ejecuta `root = s;` $\rightarrow$ `root = null;` .
  - La condición interna `if (s != null)` no se ejecuta, protegiendo contra `NullPointerException` .
  - Al regresar a `remove` , se ejecuta `n--;` $\rightarrow n = 0$. Invariantes de árbol vacío totalmente preservados.

> *No pases a la siguiente lectura hasta haber repasado este trazado de punteros en papel.*